Обновлённый проект Amaliyot / GEADEZIST

PDF результатов • Swagger • исправления ошибок

Практическое руководство по запуску и проверке. Дата подготовки: 30 октября 2026. Для демонстрации используются только синтетические данные.

Начните с готовой сборки

Распакуйте архив. В Windows запустите **run-demo.cmd**. В Linux/macOS выполните **./run-demo.sh**. Нужна установленная Java 17 или новее. Готовый JAR уже включён в папку dist, Maven для этого способа не нужен.

```
java -jar dist/Amaliyot.jar --spring.profiles.active=demo
```

Дождитесь сообщения о запуске приложения и откройте Swagger:

```
http://localhost:8080/swagger-ui.html
```

Два аккаунта деморежима

Роль	Email	Пароль
Ученик (USER)	student.demo@gmail.com	DemoUser123!
Администратор	admin.demo@example.test	DemoAdmin123!

У ученика уже есть заполненный профиль, 40 доступных вопросов, результат **38/40 = 95%** и сертификат. Можно сразу проверить PDF. Деморежим слушает localhost и хранит данные в памяти; после остановки они создаются заново.

Что находится в архиве

Исходники и Maven Wrapper, готовая сборка, инструкции, тесты, Docker Compose, CI и примеры PDF. Основные инструкции: README.md и docs/SWAGGER_GUIDE_RU.md. Подробный отчёт о выполненных проверках: docs/VERIFICATION.md.

Открыть результат в PDF

1. Войти

В Swagger раскройте **POST /api/auth/login**, нажмите Try it out и выберите пример demoUser. Или вставьте JSON ниже. Нажмите Execute; ожидается код 200.

```
{
  "email": "student.demo@gmail.com",
  "password": "DemoUser123!"
}
```

2. Вставить токен

Скопируйте **accessToken** из реального Response body, без кавычек. Нажмите Authorize и вставьте только токен, **без слова Bearer**. Нажмите Authorize, затем Close.

3. Взять ID результата

Выполните **GET /api/tests/results**. В Response body найдите content[0].id. Это attemptId. Не используйте предполагаемый ID: в вашей базе он может отличаться.

4. Получить документ

Выполните **GET /api/tests/results/{attemptId}/pdf** с полученным ID. Ожидается 200 и application/pdf. Нажмите **Download file**, затем откройте документ. В нём должны быть имя, направление, баллы, статус и даты.

```
GET /api/tests/results/{attemptId}/pdf?download=true
Authorization: Bearer <accessToken>
```

Параметр download=true задаёт attachment; без него используется inline. Swagger обычно предлагает скачать бинарный ответ. Обычная адресная строка браузера не отправляет JWT: поэтому открытие защищённого URL без заголовка Authorization вернёт 401.

Сертификат

Выполните **GET /api/certificates**, скопируйте certificateNumber и запросите **GET /api/certificates/{certificateNumber}/pdf**. Публичный адрес **GET /api/certificates/verify/{certificateNumber}** проверяет актуальный статус без входа.

В PDF встроены шрифты для русского и узбекского текста. Время выводится в Asia/Tashkent. Отозванный сертификат явно отмечается как отозванный.

Пройти новый экзамен

1. Найти направление

Выполните **GET /api/profile**. Используйте professionId как categoryId в **POST /api/tests/categories/{categoryId}/attempts**. Демо-профиль готов к запуску; новая попытка возвращает 40 вопросов и deadlineAt.

Сохраните верхний id как attemptId. Для ответов берите questions[].id и questions[].options[].id **именно из этой попытки**. В админском банке вопросов другие ID.

2. Сохранить ответ

Вызовите **PUT /api/tests/attempts/{attemptId}/questions/{questionId}/answer**. Для SINGLE выберите один ID, для MULTIPLE - несколько, для NUMERIC отправьте число.

Тип	Пример тела запроса
SINGLE	`{"selectedOptionIds": [123]}` 123 замените реальным ID варианта.
MULTIPLE	`{"selectedOptionIds": [123,124]}` Оба ID берите из вариантов одного вопроса.
NUMERIC	`{"numericValue": 12.5}` Число - ваш ответ на конкретный вопрос.
Очистка	`{"selectedOptionIds": []}` Для NUMERIC: {"numericValue": null}.

После PUT сделайте отдельный GET /api/tests/attempts/{attemptId}: ответ должен сохраниться. Не смешивайте numericValue и выбранные варианты.

3. Сохранить и продолжить

Вызовите **POST /api/tests/attempts/{attemptId}/save**, затем /resume. Ответы должны сохраниться, а deadlineAt остаться прежним. **Сохранение не останавливает таймер.**

4. Завершить

Вызовите **POST /api/tests/attempts/{attemptId}/finish**. Незаполненные ответы считаются неправильными. Повторите finish: баллы и время завершения должны остаться прежними. Затем получите PDF.

Сертификат выдаётся при 35+ правильных ответах из 40 без просрочки. PENDING означает завершённый результат, ожидающий проверки; PDF уже доступен.

Проверки, которые важнее всего

Проверяйте не только успешные запросы, но и отказы. Эти сценарии показывают, что права, сохранение и правила экзамена действительно работают.

Сценарий	Действие	Ожидание
Нет токена	Authorize → Logout; затем профиль или PDF	401, JSON
Недостаточно прав	Под USER: GET /api/admin/dashboard	403
Чужой результат	Под другим аккаунтом: PDF ученика	404
Экзамен не завершён	PDF новой активной попытки до finish	409
Неверный вариант	ID варианта другого вопроса	400
Числовой предел	Для NUMERIC отправить 1e20	400
Сохранение	PUT answer → отдельный GET attempt	Ответ сохранён
Повторное завершение	Два вызова finish	Тот же результат
Поздний ответ	PUT answer после завершения	409
Выход	POST /api/auth/logout → профиль со старым JWT	401

Проверка администратора

Войдите с admin.demo@example.test / DemoAdmin123! и **замените токен** в Authorize. GET /api/admin/dashboard должен вернуть 200. Доступны управление вопросами и просмотр попыток через /api/admin/attempts. Приватный PDF владельца не становится общедоступным из-за роли администратора.

Ограничение частых запросов

Больше 20 auth-запросов за 60 секунд с одного адреса дают **429 AUTH_RATE_LIMITED** и заголовок Retry-After. Пять неудачных входов по одному логину дополнительно включают блокировку логина на 15 минут. Эти проверки лучше делать в конце демонстрации.

Ошибки PDF возвращаются в JSON, даже когда клиент указал Accept: application/pdf. Это позволяет понять причину отказа, а не получать повреждённый документ.

Что исправлено и добавлено

Проблема	Изменение
Ошибки доступа давали 500	Корректные JSON-ответы 401/403 и отдельные обработчики HTTP-ошибок.
Просроченный тест оставался активным	EXPIRED и баллы сохраняются даже при ответе 409 на save/resume.
Время зависело от планировщика	Фиксируется фактический deadline; он же используется для пересдачи.
Запросы по одному на вопрос	Ответы и выбранные ID загружаются пакетно. Устранён риск lazy-сериализации.
Числовые значения обходили проверку	Контроль 14 целых и 6 дробных разрядов, включая экспоненциальную запись.
Гонки и PostgreSQL-блокировки	Блокировка пользователя согласована; снимки попытки читаются отдельно от блокировки её строки.
Случайный отказ генерации пароля	Новый версионированный PBKDF2-SHA256, 600 000 итераций, 256 бит. Старые хеши обновляются при успешном входе.
Потеря локальной базы при остановке	Обычный режим использует файловую H2 в data/. Память используется явно в demo.

Два дополнительных улучшения

Защита auth-запросов по IP с ограниченной таблицей и Retry-After. Она дополняет существующую защиту входа по логину.

Воспроизводимый деморежим с двумя аккаунтами, 40 вопросами, готовым результатом и сертификатом. Он помогает проверить весь API без ручного заполнения базы.

Совместимость

Сохранены Java 17, Spring Boot 4.1.1, springdoc 3.1.1 и старые маршруты API. PDFBox 3.0.8 добавлен для PDF. Миграции V1-V13 не менялись. Исходный .env и IDE-метаданные не включены в поставку; .env.example очищен от готовых секретов.

Сборка и дальнейшая работа

Из исходников

```
.\mvnw.cmd clean verify
java -jar target/Amaliyot-0.0.1-SNAPSHOT.jar --spring.profiles.active=demo
```

Первой сборке нужен интернет. Изменения исходников попадают в новый JAR в target; приложенный dist/Amaliyot.jar автоматически не обновляется. Тесты не читают .env и используют отдельную базу.

Автоматическая HTTP-проверка демо

```
python scripts/smoke_demo.py
```

Нужен Python 3 и запущенное demo. Скрипт обращается только к localhost, создаёт дополнительную демопопытку, проверяет API и сохраняет PDF/OpenAPI в docs/examples. JWT в файлы не записываются.

Обычный запуск и PostgreSQL

Для сохранения данных используйте обычный локальный режим или PostgreSQL. Скопируйте .env.example в .env и заполните параметры своего окружения. В prod секрет JWT обязателен, Swagger по умолчанию выключен.

```
docker compose up --build -d
docker compose logs -f app
```

Compose запускает PostgreSQL 16 и приложение, сохраняет БД и загрузки в именованные тома. Для локальной проверки Swagger в Compose задайте SWAGGER_ENABLED=true. Обычная остановка: docker compose down. Ключ -v удаляет тома и данные.

Что проверено

45 Java-тестов на H2 и **28 интеграционных проверок** на PostgreSQL 16.15 прошли без ошибок. Финальный JAR запущен; выполнено **26 HTTP-запросов**, PDF скачаны и проверены визуально. Все 13 старых миграций сохранены. Подробности: docs/VERIFICATION.md.

Оставшиеся подключения

Восстановление пароля требует настоящего SMS/email-провайдера и сейчас возвращает 503. Для публичного размещения нужны ваш хостинг, HTTPS и reverse proxy. При нескольких экземплярах приложения IP-лимит должен быть общим. Docker-образ и GitHub workflow приложены; факт их запуска указан отдельно в отчёте.

Официальные источники и подробности

springdoc: https://springdoc.org/
Apache PDFBox: https://pdfbox.apache.org/download.html
Spring Boot: https://docs.spring.io/spring-boot/
Полные инструкции: README.md, docs/SWAGGER_GUIDE_RU.md, docs/CHANGES_RU.md.